

Synapsa Project Architecture & Technical Documentation

1. Project Overview

Synapsa is a comprehensive web-based platform designed as a cognitive companion and memory-enhancing application. It integrates gamified cognitive exercises, caregiver monitoring, and a state-of-the-art 3D AI companion named **NOVA**. The platform is built with a focus on accessibility, high-performance 3D rendering, and smooth, cinematic user experiences.

2. Frontend Architecture

The frontend is built as a highly interactive Single Page Application (SPA).

- **Core Framework:** React 18, utilizing functional components and hooks.
- **Build Tool:** Vite for extremely fast HMR (Hot Module Replacement) and optimized production bundling.
- **Language:** TypeScript for strict type-checking and robust interfaces (e.g., `CaregiverData`, `AIOutput`).
- **Routing:** `react-router-dom v6` for seamless client-side navigation.
- **Animations:** `framer-motion` handles page transitions (`<AnimatePresence>`), micro-interactions, and UI animations.
- **Styling:** Vanilla CSS using CSS variables (`--color-tea-green`, `--bone`, `--bg-1`) for a unified design system. The aesthetic utilizes modern UI trends like glassmorphism (translucent backgrounds with blur filters) and fluid typography.

3. The 3D Engine & WebGL (NOVA)

Synapsa features a rich, continuous 3D environment that runs efficiently across the entire application.

- **Library:** `react-three-fiber (R3F)` bridges React and Three.js.
- **Global Canvas:** A single `<Canvas>` is mounted at the root (`App.tsx`) to manage the WebGL context. This ensures that 3D assets are loaded once and the render loop is never destroyed during page navigation.
- **Environment:** `GlobalEnvironment3D.tsx` renders ambient particles, glowing orbs, and lighting that persists globally.
- **VoiceOrb3D (NOVA):** The AI avatar is a complex procedural shader object (utilizing `MeshTransmissionMaterial` for glass-like refractions and Perlin noise for organic deformations). It is injected into the global scene conditionally when navigating to the Companion page, eliminating context-loss bugs and maintaining 60 FPS.

4. Backend Architecture & AI API

The backend serves as a secure proxy to interact with Google's Generative AI while keeping API keys hidden from the client.

- **Local Development:** An Express server (`server.js`) handles POST requests at `/api/chat`.
- **Production (Vercel):** The application is deployed on Vercel, utilizing Vercel Serverless Functions (`api/chat.js`) configured via `vercel.json`.
- **AI Model:** Google Gemini (`gemini-2.5-flash`) powered by `@google/genai`.
- **System Prompt:** NOVA is configured via a strict `SYSTEM_PROMPT` to act as a warm, concise cognitive companion. The backend dynamically appends user context (Streak, Level, XP) to the prompt.
- **Grounding / Search:** The API analyzes user input to detect temporal queries (e.g., "latest news", "president of USA"). If detected, it enables Google Search tools (`googleSearch: {}`) within the Gemini config to fetch real-time grounded data, returning both the response and citation sources to the frontend.

5. Core Features & Pages

1. **NOVA Companion Page:** A text-based chat interface allowing users to converse with the AI. Includes quick-reply suggestions and a sleek translucent UI overlaid on the 3D canvas.
2. **Cognitive Games:**
 - *Memory Match & Pattern Games:* Dynamically generated levels using `GameService.ts`. Cultural objects (e.g., Jaapi, Kopou Phool) are utilized for localization and familiarity.
3. **Caregiver Dashboard:** A dedicated portal providing analytics, adherence tracking, and cognitive decline/improvement charts using mock backend data integration (`AnalyticsService`).
4. **Authentication:** JWT-style mock `AuthService` handling simulated login, signup, and user sessions.

6. Services Layer

The application abstracts business logic into dedicated services (`src/services/`):

- **VoiceService & AudioService:** Handles browser-based SpeechSynthesis (Text-to-Speech) and procedural Web Audio API sounds (hover pops, reward chimes, ambient drones).
- **DatabaseService & SyncService:** Manages local storage persistence and offline detection, structuring data syncing patterns for future cloud database integration.

7. Deployment Configuration

- **Vercel Config:** `vercel.json` maps `/api/(.*)` to `/api/index.js` (or `chat.js`), ensuring seamless routing between the static Vite frontend and the serverless Node.js backend.
- **Environment Variables:** `GEMINI_API_KEY` is securely stored in Vercel's environment variables, preventing client-side exposure.